



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS
POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 5

Uso de los Pines de Propósito General

de Entrada/Salida en Raspberry Pi Pico W

Carrera: Ingeniería Mecatrónica – Cuarto Semestre

Alumno: Saúl Martínez Almazán

Matrícula: 383912

Docente: Ing. Luis Jesús Padrón Padrón

Fecha: 24 de Febrero de 2026

Índice

1. Introducción	2
2. Desarrollo de la Práctica	2
2.1. Materiales y Conexiones	2
2.2. Paso a Paso del Proceso	3
3. Preguntas de Repaso	3
3.1. Pasos para configurar un pin GPIO como salida	3
3.2. Cálculo del valor de resistencia para el LED rojo	4
3.3. Valor comercial de resistencia más óptimo	4
3.4. ¿Qué es el PWM y cuál es su función con LEDs?	5
4. Conclusiones	5
5. Anexos	6
5.1. Código C++ (main.cpp)	6
5.2. Archivo CMakeLists.txt	6
5.3. Evidencia de Compilación en Terminal	7
5.4. Diagrama Esquemático (Fritzing)	8
5.5. Evidencia en Protoboard	9

1. Introducción

Los microcontroladores modernos cuentan con pines de propósito general de entrada/salida, conocidos como GPIO (*General Purpose Input/Output*), los cuales permiten interactuar con el mundo físico mediante señales digitales. En el caso del Raspberry Pi Pico W, basado en el microcontrolador RP2040, se dispone de hasta 26 pines GPIO accesibles al usuario.

En esta práctica se explora el uso básico de dichos pines configurándolos como salida digital para controlar el encendido y apagado de un LED físico conectado al pin GP3. A diferencia de la práctica anterior, donde se utilizó el LED integrado del Pico W a través del chip de Wi-Fi CYW43439, en esta ocasión se trabaja directamente con el hardware externo a través del bus GPIO del RP2040, haciendo uso del SDK de C/C++.

El dominio de los pines GPIO constituye la base de cualquier sistema embebido: desde el control de actuadores y la lectura de sensores, hasta la implementación de protocolos de comunicación como I2C, SPI y UART.

2. Desarrollo de la Práctica

2.1. Materiales y Conexiones

Para la realización de esta práctica se utilizaron los siguientes componentes:

- Raspberry Pi Pico W
- LED verde de 5 mm
- Resistencia de **220 Ω** (valor comercial seleccionado)
- Protoboard
- Alambre calibre 22
- Cable micro-USB

Esquema de conexión:

- El ánodo del LED se conectó al pin **GP3** del Pico W.
- La resistencia se colocó en serie entre GP3 y el ánodo del LED.
- El cátodo del LED se conectó a un pin **GND** del Pico W.

2.2. Paso a Paso del Proceso

1. **Diseño del circuito:** Se calculó el valor de resistencia adecuado para limitar la corriente al LED rojo (ver Sección 3). Se ensamblaron los componentes en la proto-board según el diagrama esquemático.
2. **Configuración del entorno:** Se abrió el proyecto en VS Code con el SDK de Raspberry Pi Pico previamente instalado. Se verificó que la variable de entorno `PICO_SDK_PATH` estuviese correctamente configurada.
3. **Estructura del proyecto:** Se creó la siguiente organización de archivos:
 - `main.cpp` — Código fuente principal en C++
 - `CMakeLists.txt` — Archivo de configuración de compilación
 - `build/` — Carpeta generada para los binarios
4. **Escritura del código:** Se implementó el programa en `main.cpp` utilizando las funciones del SDK para inicializar y controlar el pin GP3 (ver Anexo 5.1).
5. **Compilación:** Desde la terminal *Pico Developer PowerShell*, dentro de la carpeta `build/`, se ejecutaron los comandos:
 - `cmake ..` — para generar los archivos de construcción
 - `nmake` — para compilar y generar el archivo `.uf2`
6. **Carga del programa:** Se mantuvo presionado el botón BOOTSEL del Pico W al conectarlo por USB, montándolo como unidad de almacenamiento. El archivo `.uf2` generado se arrastró a dicha unidad.
7. **Verificación:** Tras la carga, el LED físico conectado a GP3 comenzó a parpadear con un periodo de 500 ms (250 ms encendido, 250 ms apagado), confirmando el funcionamiento correcto del programa.

3. Preguntas de Repaso

3.1. Pasos para configurar un pin GPIO como salida

Para configurar un pin GPIO como salida digital en el Raspberry Pi Pico W mediante el SDK de C/C++, se deben seguir los siguientes pasos:

1. **Incluir la librería estándar:** Agregar `#include "pico/stdlib.h"` al inicio del archivo fuente para tener acceso a las funciones de GPIO.
2. **Definir el número de pin:** Declarar una constante con el número del pin GPIO que se desea utilizar. En esta práctica se utilizó GP3:

```
const uint LED_PIN = 3;
```

3. **Inicializar el pin:** Llamar a la función `gpio_init(LED_PIN)` para habilitar el pin y configurarlo en su estado por defecto.
4. **Establecer la dirección:** Llamar a `gpio_set_dir(LED_PIN, GPIO_OUT)` para definir el pin como salida digital. La constante `GPIO_OUT` indica dirección de salida.
5. **Controlar el estado:** Utilizar `gpio_put(LED_PIN, 1)` para poner el pin en estado alto (3.3 V) y `gpio_put(LED_PIN, 0)` para ponerlo en estado bajo (0 V).

3.2. Cálculo del valor de resistencia para el LED rojo

Para determinar la resistencia necesaria se aplica la **Ley de Ohm** junto con el análisis del circuito en serie formado por el pin GPIO, la resistencia y el LED.

Datos del problema:

- Voltaje de salida del pin GPIO: $V_{CC} = 3,3 \text{ V}$
- Caída de voltaje en el LED verde: $V_{LED} = 2,2 \text{ V}$
- Corriente deseada: $I = 15 \text{ mA} = 0,015 \text{ A}$

Procedimiento:

El voltaje que cae sobre la resistencia es la diferencia entre el voltaje de alimentación y la caída del LED:

$$V_R = V_{CC} - V_{LED} = 3,3 \text{ V} - 2,2 \text{ V} = 1,1 \text{ V}$$

Aplicando la Ley de Ohm ($V = IR$), se despeja la resistencia:

$$R = \frac{V_R}{I} = \frac{1,1 \text{ V}}{0,015 \text{ A}} = 73,3 \Omega$$

Resultado: El valor ideal de resistencia es $R = 73,3 \Omega$.

3.3. Valor comercial de resistencia más óptimo

Los valores comerciales de resistencia siguen la serie E24 (tolerancia del 5 %). Los valores más cercanos a los $73,3 \Omega$ calculados son:

Cuadro 1: Comparación de valores comerciales cercanos a $73,3 \Omega$

Valor comercial	Corriente resultante	Error respecto a 15 mA	¿Adecuado?
100 Ω	18.0 mA	+3.0 mA	Riesgo de daño
120 Ω	15.0 mA	0.0 mA	Ideal (no siempre disponible)
150 Ω	12.0 mA	-3.0 mA	Ligeramente menor
220 Ω	8.2 mA	-6.8 mA	Seguro y común

El valor comercial más óptimo seleccionado fue **220 Ω** , ya que:

- Es el valor estándar más disponible en cualquier laboratorio o tienda.
- Garantiza que la corriente se mantenga por debajo del límite máximo del pin GPIO del RP2040 (12 mA por pin) y del LED.
- Produce una luminosidad visible y adecuada para propósitos de prueba.
- Protege tanto el LED como el microcontrolador de posibles daños por sobrecorriente.

3.4. ¿Qué es el PWM y cuál es su función con LEDs?

El **PWM** (*Pulse Width Modulation*, o Modulación por Ancho de Pulso) es una técnica digital que consiste en generar una señal cuadrada de frecuencia fija pero con un *ciclo de trabajo* (*duty cycle*) variable. El ciclo de trabajo representa el porcentaje del periodo en que la señal permanece en estado alto.

$$\text{Duty Cycle} = \frac{t_{ON}}{T} \times 100\%$$

donde t_{ON} es el tiempo en estado alto y T es el periodo total de la señal.

Función del PWM en el control de LEDs:

- **Control de brillo:** Al variar el ciclo de trabajo, se controla el promedio de potencia entregada al LED, lo que permite regular su intensidad luminosa sin cambiar el voltaje de alimentación. Un duty cycle del 10 % produce un LED casi apagado, mientras que uno del 90 % lo enciende casi al máximo.
- **Eficiencia energética:** El LED siempre recibe su voltaje nominal cuando está encendido, evitando pérdidas por disipación en resistencias variables.
- **Compatibilidad digital:** Permite controlar dispositivos analógicos (como el brillo de un LED) desde una salida puramente digital, lo cual es fundamental en microcontroladores como el RP2040.

El Raspberry Pi Pico W incorpora un módulo PWM de 16 bits con 8 canales independientes, capaz de controlar hasta 16 pines simultáneamente con resoluciones de hasta 65536 niveles de brillo.

4. Conclusiones

La práctica permitió consolidar los conocimientos fundamentales sobre el manejo de pines GPIO en el Raspberry Pi Pico W mediante el SDK de C/C++. A diferencia del control del LED integrado, que requería la inicialización del chip CYW43439, el control de hardware externo a través de los pines GPIO del RP2040 resulta más directo y accesible, haciendo uso de únicamente tres funciones del SDK: `gpio_init()`, `gpio_set_dir()` y `gpio_put()`.

El cálculo previo de la resistencia de protección resultó esencial para garantizar la integridad tanto del LED como del microcontrolador. El valor de $220\,\Omega$ demostró ser una elección segura y práctica para este tipo de circuitos de prueba.

Esta práctica sienta las bases para el desarrollo de proyectos más complejos, como el sistema de visión “*The Sentinel*”, en el que el control preciso de actuadores externos será un componente fundamental del sistema de respuesta.

5. Anexos

5.1. Código C++ (main.cpp)

```
1 #include "pico/stdlib.h"
2
3 int main(){
4     const uint LED_PIN = 3;
5
6     gpio_init(LED_PIN);
7     gpio_set_dir(LED_PIN, GPIO_OUT);
8
9     while(true){
10         gpio_put(LED_PIN, 1);
11         sleep_ms(250);
12         gpio_put(LED_PIN, 0);
13         sleep_ms(250);
14     }
15 }
```

5.2. Archivo CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.13)
2
3 include($ENV{PICO_SDK_PATH}/external/pico_sdk_import.cmake)
4
5 project(led_gpio C CXX ASM)
6 set(CMAKE_C_STANDARD 11)
7 set(CMAKE_CXX_STANDARD 17)
8
9 pico_sdk_init()
10
11 add_executable(led_gpio
12     main.cpp
13 )
14
15 pico_add_extra_outputs(led_gpio)
```

```
16
17 target_link_libraries(led_gpio
18     pico_stdlib
19 )
```

5.3. Evidencia de Compilación en Terminal

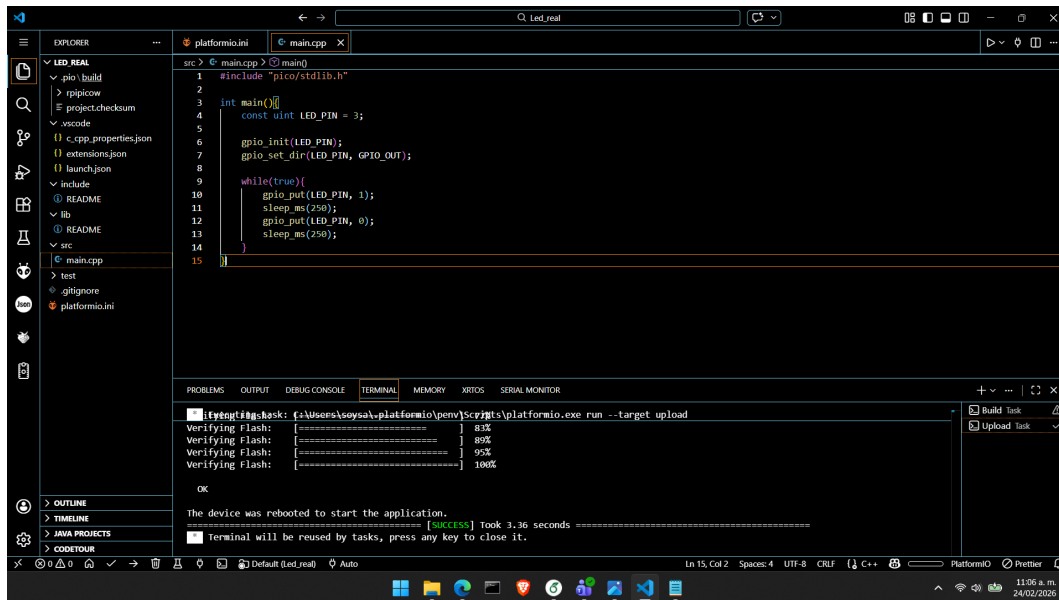


Figura 1: Captura de pantalla de la terminal mostrando el proceso de compilación con cmake y nmake del proyecto led_gpio

5.4. Diagrama Esquemático (Fritzing)

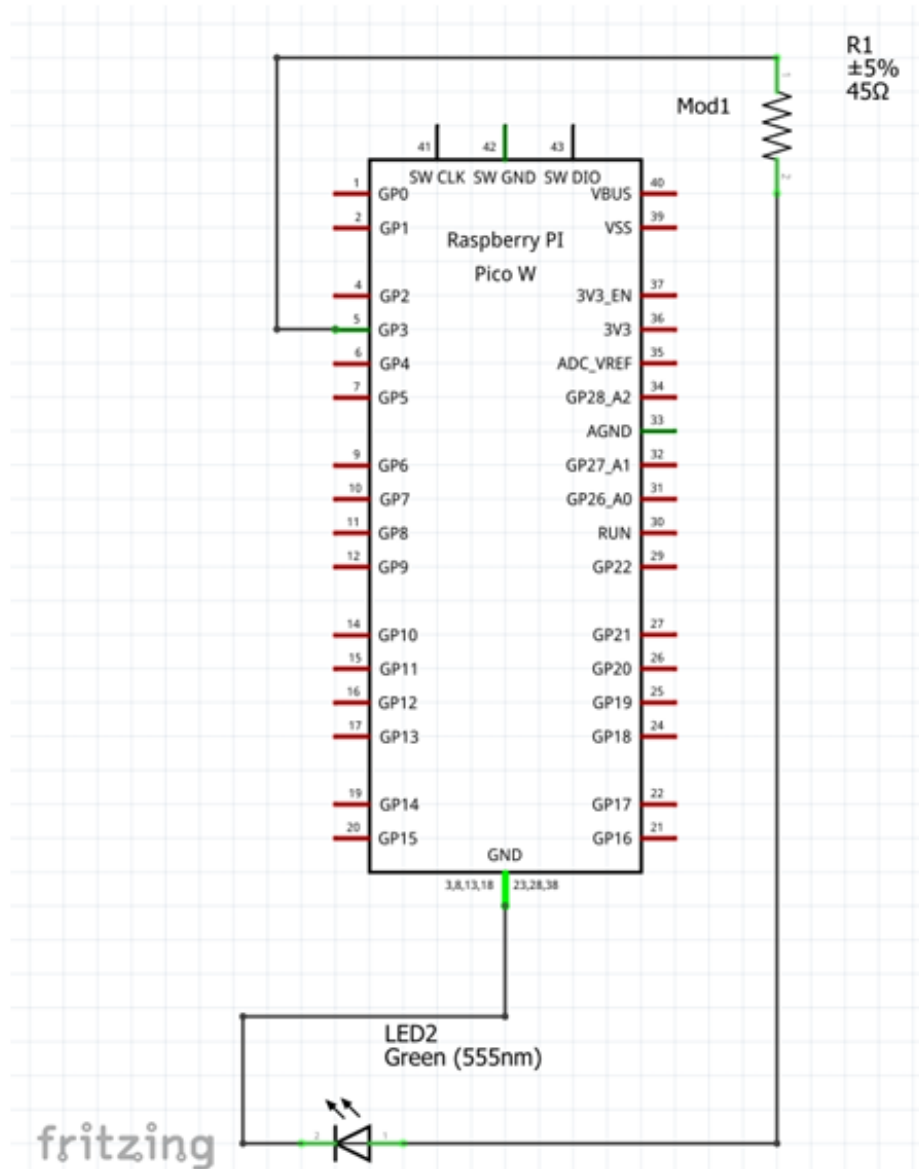


Figura 2: Diagrama esquemático del circuito: LED verde con resistencia de $220\ \Omega$ conectado al pin GP3 del Raspberry Pi Pico W

5.5. Evidencia en Protoboard

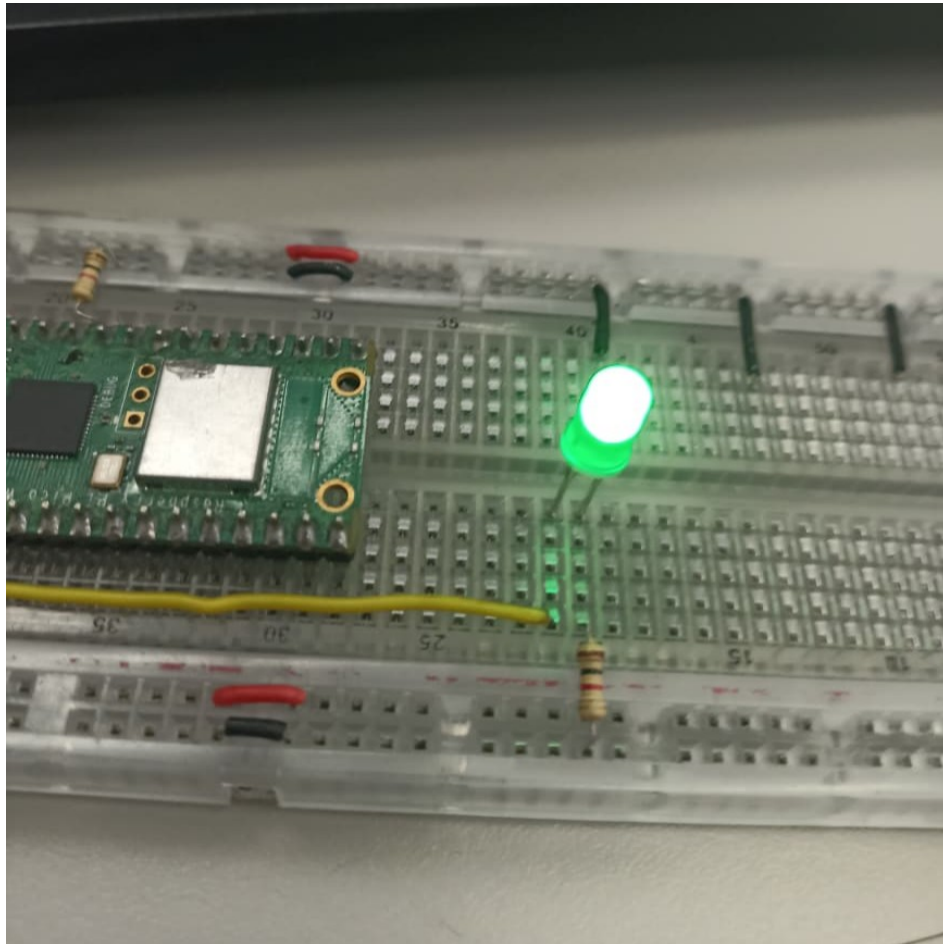


Figura 3: Vista del montaje en protoboard del circuito de control de LED con Raspberry Pi Pico W

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.